Question (from image_f04253.png):

- A program frequently accesses a few recent variables and iterates over large arrays. Predict which parts benefit from temporal and spatial locality, and justify your reasoning.

Answer:

- **Part Benefiting from Temporal Locality:** The **recent variables**.
  - *Reasoning:* Temporal locality states that data accessed once is highly likely to be accessed again in the near future. Since the program *frequently* reuses these specific variables, keeping them in the cache ensures instant re-access without fetching from RAM again.
- **Part Benefiting from Spatial Locality:** Iterating over **large arrays**.
  - *Reasoning:* Spatial locality states that if a data item is accessed, items stored at nearby memory addresses are highly likely to be accessed soon. Array elements are stored contiguously (back-to-back) in memory, so loading the first element fetches the subsequent elements into the cache block automatically, preventing future misses.

Question (from image_f04559.png):

- How does spatial locality be helpful in handling cache misses? Explain with example.

Answer:

- **How it helps:** Spatial locality means that when the CPU accesses a specific memory location, it is highly likely to access nearby memory locations soon after.
  Cache memory exploits this by fetching an **entire multi-byte data block (line)** from the main memory instead of fetching just the single requested byte/word during a cache miss. By bringing neighboring data into the cache ahead of time, subsequent requests for nearby data result in immediate **cache hits**, avoiding further misses.
- **Example:** Consider looping through a 1D integer array: `arr[0]`, `arr[1]`, `arr[2]`, `arr[3]`.
  1. When the CPU requests `arr[0]`, it results in a **cache miss** because the data isn't there yet.
  2. Instead of loading *only* `arr[0]`, the system fetches a full cache block containing `arr[0]`, `arr[1]`, `arr[2]`, and `arr[3]` together from RAM into the cache.
  3. When the loop moves on to process `arr[1]`, `arr[2]`, and `arr[3]`, the CPU finds them already waiting inside the cache, turning those subsequent accesses into fast **cache hits**.

Question (from image_f04996.png):

- Locality

Answer:

- **What it is:** The Principle of Locality (or Locality of Reference) is the tendency of a computer program to access the same memory locations repeatedly, or nearby memory locations, within a short period of time.
- **Types of Locality:**
  - **Temporal Locality:** If a specific memory location is accessed once, it is highly likely to be accessed again in the near future (e.g., variables inside a loop).
  - **Spatial Locality:** If a specific memory location is accessed, locations physically close to it are highly likely to be accessed soon after (e.g., stepping through an array sequentially).